

MOBILE APPLICATION AND DEVELOPMENT – WEEK 2

Hello World App & Deploying on AVD

Building your first Android project · Project structure · Running it on an emulator and a real device

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science

TODAY'S SESSION – THEORY 1

Agenda

- 01** Creating your first project and touring what Android Studio generates
- 02** What happens when you press Run — Gradle, the APK, and installation
- 03** The Android Virtual Device (AVD) — creating one and running your app on it
- 04** Running on a real device, reading Logcat, and fixing common first-run errors

LAST WEEK

From Setup to a Real App

Week 1 installed Android Studio and the SDK, and covered the Java OOP you'll lean on constantly — classes, objects, constructors. Today we stop reading about Android and start building in it.

Then

Tools installed, environment configured

Now

A working app you built, running on a screen

PART 1 – BUILDING HELLO WORLD

File → New → New Project

Choose the "Empty Views Activity" template — the simplest starting point, with one screen and no extra boilerplate.

Name Your app's display name, e.g. "My First App"	Package name A unique reverse-domain id, e.g. com.diu.myfirstapp	Language Java (what this course uses) or Kotlin	Minimum SDK The oldest Android version your app will still install on
---	--	---	---

MAKING SENSE OF THE NUMBERS

API Levels & SDK Versions

Every Android release has a version name and a numeric API level. Code checks the number, not the name – "Android 14" and "API 34" refer to the same release.

minSdk	targetSdk	compileSdk
Oldest version the app will install on	Version it's tested and optimized for	Version its code is compiled against

A lower minSdk reaches more devices but means fewer newer APIs are safely available without extra checks.

WHAT THE WIZARD JUST CREATED

The Project Tree, at a Glance

```
app/  
  manifests/      AndroidManifest.xml  
  java/           MainActivity.java  
  res/  
    layout/       activity_main.xml  
    values/        strings.xml, colors.xml  
    drawable/      icons, shapes  
    mipmap/        launcher icons  
  build.gradle    module-level config
```

We'll look at each of these in turn — this is the shape of every Android project you build this semester.

THE APP'S ID CARD

AndroidManifest.xml

```
<application android:label="My First App" android:icon="@mipmap/ic_launcher">
  <activity android:name=".MainActivity" android:exported="true">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
</application>
```

The MAIN/LAUNCHER intent-filter is what makes this the screen that opens when the user taps your app icon. Every Activity must be declared here before Android will run it.

THE SCREEN'S CONTROLLER

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

`onCreate()` runs once when the screen is first built. `setContentView()` is what connects this Java class to its XML layout — everything you see on screen comes from that one line.

THE SCREEN'S APPEARANCE

activity_main.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

One TextView, centered by four constraints pinning it to all four edges of its parent. We'll cover ConstraintLayout properly in a later class — for now, just recognize the shape.

CONNECTING JAVA TO A VIEW

findViewById()

A View declared in XML is just a description until Java gets a reference to the actual object on screen — that's what `findViewById()` does, using an id you assign in the layout.

```
// in activity_main.xml: android:id="@+id/greetingText"

TextView greeting = findViewById(R.id.greetingText);
greeting.setText("Hello, Android!");
```

Call it after `setContentView()` in `onCreate()` — the layout has to be inflated first, or there's nothing yet to find.

PUTTING IT TOGETHER

Changing Text at Runtime

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        TextView greeting = findViewById(R.id.greetingText);  
        greeting.setText(getString(R.string.greeting) + "!");  
    }  
}
```

Note `getString(R.string.greeting)` — reading from `strings.xml` in Java, the same resource used in XML with `@string/greeting`.

EVERY NON-CODE ASSET LIVES HERE

Inside res/

layout/

One XML file per screen, describing its Views

values/

strings.xml, colors.xml, themes.xml
— text and design tokens

drawable/

Images, icons, and XML-defined shapes/gradients

mipmap/

The launcher icon, at several screen densities

Reference any resource from code or XML with `R.type.name` — e.g. `R.layout.activity_main`, `R.string.app_name`.

NEVER HARDCODE TEXT

strings.xml

```
<resources>
  <string name="app_name">My First App</string>
  <string name="greeting">Hello World!</string>
</resources>
```

Used as `@string/greeting` in XML, or `getString(R.string.greeting)` in Java. One place to translate, rename, or update every piece of text — instead of hunting through layout files.

THE BUILD RECIPE

build.gradle (Module: app)

```
android {  
    namespace 'com.diu.myfirstapp'  
    compileSdk 34  
    defaultConfig {  
        applicationId "com.diu.myfirstapp"  
        minSdk 24  
        targetSdk 34  
    }  
}  
  
dependencies {  
    implementation 'androidx.appcompat:appcompat:1.7.0'  
}
```

Every external library your app uses — including ones we'll add later, like Room and Firebase — is declared as one line here.

A QUICK REFRESHER FROM WEEK 1

Tools → SDK Manager

If your project's `compileSdk` or `targetSdk` isn't installed, Android Studio will ask you to download it here before it can build.

SDK Platforms tab

Which Android versions you can build against and emulate

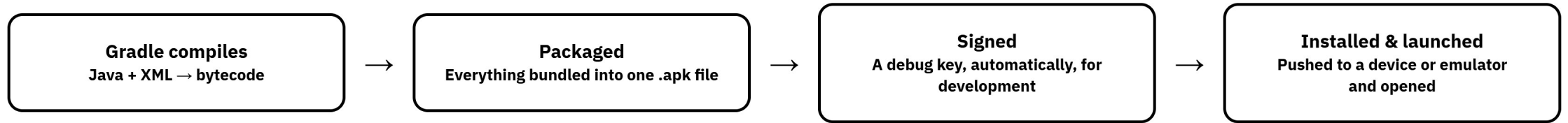
SDK Tools tab

Build tools, platform tools (adb), and emulator binaries

Only install the API levels you actually need — each one takes several gigabytes.

WHAT "RUN" ACTUALLY DOES

From Source Code to a Running App



All four steps happen automatically every time you click the green Run button — a "Gradle sync" is this process resolving dependencies before the build even starts.

PART 2 – RUNNING IT

What Is an AVD?

An Android Virtual Device is a software simulation of a phone or tablet that runs on your computer — no physical hardware required to test your app.

Hardware profile

Screen size, RAM, and resolution it simulates

System image

The actual Android OS version it boots

The emulator

The program that actually runs the AVD

TOOLS → DEVICE MANAGER

Creating Your First Virtual Device

1 Click "Create Device" and pick a hardware profile — Pixel 6 is a safe default

2 Choose a system image — download one matching your app's target SDK

3 Name it and finish — it now appears in your list of available devices

Do this once — the AVD is saved and reused for every future run, not recreated each time.

BOTH HAVE THEIR PLACE

Emulator vs. Real Device

AVD (Emulator)

No cable needed, easy to test many screen sizes/OS versions, but slower and uses a lot of RAM

Physical device

Real performance, real sensors and camera, but only tests that one specific device

Use the emulator for everyday development; test on a real device before you consider a feature finished — especially anything touching the camera, GPS, or performance.

WORTH KNOWING ON DAY ONE

The Emulator's Side Panel

Rotate

Switch between portrait and landscape — test both

Volume / Power

Hardware buttons, same as a real phone

Extended controls (...)

Fake GPS location, battery level, incoming calls, and more

We'll use the fake GPS location panel directly when we reach the Google Maps unit later in the course.

THE MOMENT OF TRUTH

Deploying to the Emulator

- 1 Select your AVD from the device dropdown in the toolbar
- 2 Click the green Run ► button (or Shift+F10)
- 3 Wait for the emulator to boot — the first launch can take a few minutes
- 4 Your app installs and launches automatically once it's ready

Leave the emulator running between runs — later launches reuse it and are much faster.

USING YOUR OWN PHONE

Running on a Real Device

- 1 **Settings → About Phone → tap "Build Number" 7 times to unlock Developer Options**
- 2 **Settings → Developer Options → enable "USB Debugging"**
- 3 **Connect via USB, then tap "Allow" on the debugging prompt that appears on the phone**
- 4 **Your device now appears in Android Studio's device dropdown, same as an AVD**

THE TERMINAL, NOT JUST THE IDE

A First Look at adb

Android Debug Bridge is the command-line tool behind everything Android Studio's Run button does for you. Useful to know as we start using the terminal more next class.

```
adb devices           // list connected emulators/devices
adb install app-debug.apk // install an APK manually
adb logcat            // stream logs from the terminal
```

Android Studio's Run and Logcat panels are just a friendlier face on these same commands.

YOUR WINDOW INTO WHAT'S HAPPENING

Reading Logcat

The Logcat panel (View → Tool Windows → Logcat) streams every log message and crash from the running app in real time.

```
Log.d("MainActivity", "onCreate called");  
Log.e("MainActivity", "Something went wrong", exception);
```

Filter by your app's package name and log level (Verbose, Debug, Info, Warn, Error) to cut the noise — Error is where crashes show up first.

BEYOND PRINT STATEMENTS

Setting a Breakpoint

Click the gutter next to a line number to set a breakpoint, then run with the bug icon (Debug, not Run). Execution pauses right there, and you can inspect every variable's current value.

Step Over

Run this line, stop at the next one

Step Into

Follow execution inside a method call

Resume

Continue running until the next breakpoint

VERY LIKELY TO HAPPEN THIS WEEK

Common First-Run Problems

Gradle sync failed

Usually a network issue fetching dependencies — check your connection and retry

Emulator won't start / freezes

Not enough free RAM — close other apps, or lower the AVD's allocated RAM

"SDK location not found"

Android Studio can't find the SDK path — File → Project Structure → set the SDK location

Device not showing in the dropdown

USB debugging not enabled, or the "Allow" prompt on the phone was missed

PART 3 – WRAP-UP

Making a Small Change

Edit the text in `activity_main.xml` or `strings.xml`, then use **Apply Changes** (the lightning-bolt icon) instead of a full Run — it pushes just the change to the already-running app, in seconds.

Full Run ▶

Rebuilds and reinstalls everything — slower, always safe

Apply Changes ⚡

Patches the running app — fast, but doesn't work for every kind of change

A FINISHING TOUCH

Customizing the Launcher Icon

Right-click res/ → New → Image Asset. Android generates every required size and shape (round, square, adaptive) automatically from one source image.

The generated files replace ic_launcher in mipmap/ — the same resource the Manifest already points to, so nothing else needs to change.

TRY IT NOW – 10 MINUTES

In-Class Exercise

Starting from your Hello World project:

- 1 Change the greeting text using strings.xml, not a hardcoded string**
- 2 Increase the text size and change its color in activity_main.xml**
- 3 Run it on the AVD, then on a real device if you have one available**

BEFORE YOU MOVE ON

A Quick Checklist

1 Can you name what's in manifests/, java/, and res/ without looking?

2 Do you know which line connects an Activity to its layout XML?

3 Have you created and successfully run an app on an AVD?

4 Do you know where to look (Logcat) when something crashes?

BEFORE NEXT CLASS

What to Do This Week

Rebuild it from scratch

Create a brand-new project on your own,
without following along

Make two AVDs

Different screen sizes — see how the same
layout adapts

Cause a crash on purpose

Then find it in Logcat — get comfortable
reading a stack trace

Next lecture (Theory 2): Git for Android Development — version control for the project you just built.

Questions?

Project Structure · The Build Process · AVDs · Logcat

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science